Department of CSE (AIML), PES University
UE26CS151A -Python for Computational Problem Solving
Laboratory Week 4

1) Easy (10 Marks) — Alphabet–Digit Separator

Topics practiced: *input/output, variables, strings, lists, if/else, for loop*

Problem Statement

Given a string `s` containing only English letters (uppercase or lowercase) and digits (`0–9`):

1. Count the letters.
2. Count the digits.
3. Rearrange the string so that all letters come first, then all digits. The letters keep their original order, and so do the digits.

Print the letter count, the digit count, and the rearranged string, each on its own line.

Requirements / Conditions

- `s` has at least 1 character.
- `s` contains only English letters and digits.
- Allowed: `input()`, `print()`, variables, `type()`, `id()`, operators and expressions, control structures (`if`/`elif`/`else`, `for`, `while`), strings (indexing and +), and lists (`[]` and `.append()`).
- Not allowed: user-defined functions, dictionaries, sets, `len()`, and any string/list method other than `.append()` (for example `.split()`, `.join()`, `.sort()`).
- `print(x, end="")` prints `x` without moving to a new line.

Examples

Example 1

```
Input:
a1b2c3
Output:
3
3
abc123
```

Explanation: The letters are `a`, `b`, `c` and the digits are `1`, `2`, `3`. Letters first, then digits, gives `abc123`.

Example 2

```
Input:
12345
Output:
0
5
12345
```

Explanation: There are no letters, so the string stays the same.

Test Cases

Test Case	Input	Output
1	a1b2c3	3 3 abc123
2	py7th9on2	6 3 python792
3	12345	0 5 12345
4	abcde	5 0 abcde
5	xyz999	3 3 xyz999
6	1a2b3c4d5e	5 5 abcde12345
7	a	1 0 a

Boilerplate Code

Students write their code where the comment says so.

```
s = input()

letters = []
digits = []

letter_count = 0
digit_count = 0

# Write your code here



print(letter_count)
print(digit_count)

for ch in letters:
    print(ch, end="")

for ch in digits:
    print(ch, end="")

print()
```

Complete Solution

```
s = input()

letters = []
digits = []

letter_count = 0
digit_count = 0

for ch in s:
    if (ch >= 'a' and ch <= 'z') or (ch >= 'A' and ch <= 'Z'):
        letters.append(ch)
        letter_count = letter_count + 1
    else:
        digits.append(ch)
        digit_count = digit_count + 1

print(letter_count)
print(digit_count)

for ch in letters:
```

```
    print(ch, end="")

for ch in digits:
    print(ch, end="")

print()
```

2) Medium (15 Marks) — Weighted Password Drift Score

Topics practiced: *input/output, variables, operators and expressions, boolean logic, strings (indexing), if/elif/else, for loop*

Problem Statement

You are given two strings `A` and `B` of the same length. For every position `i` (**0-based**) where `A[i] != B[i]`, the mismatch has a **cost**:

Characters at position i	Cost
both are vowels	`i + 1`
one is a vowel and the other is a consonant	`2 * (i + 1)`
both are consonants	`3 * (i + 1)`

The vowels are `a`, `e`, `i`, `o`, `u`. Every other letter is a consonant. The order of the two characters does not matter (`a` → `b` costs the same as `b` → `a`).

Print, each on its own line:

1. The number of mismatched positions.
2. The total cost of all mismatches.
3. The position of the mismatch with the **highest cost**. If several mismatches share the highest cost, print the **earliest** position. If there are no mismatches, print `-1`.
4. The cost at that position (`0` if there are no mismatches).

Requirements / Conditions

- `A` and `B` have the same length, at least 1.
- Both contain only lowercase English letters.
- Allowed: `input()`, `print()`, variables, `type()`, `id()`, operators and expressions, control structures (`if`/`elif`/`else`, `for`, `while`), strings (indexing and `+`), and lists (`[]` and `.append()`).
- Not allowed: user-defined functions, dictionaries, sets, `len()`, and any string/list method other than `.append()` (for example `.split()`, `.join()`, `.sort()`).

Examples

Example 1

```
Input:
hello
jelly
Output:
2
13
4
10
```

Explanation: Position 0: `h` and `j` are both consonants, so the cost is 3 × 1 = 3. Position 4: `o` is a vowel and `y` is a consonant, so the cost is 2 × 5 = 10. There are 2 mismatches, the total is 3 + 10 = 13, and the highest cost is 10 at position 4.

Example 2

```
Input:
aemp
ximp
Output:
2
4
0
2
```

Explanation: Position 0: `a` (vowel) and `x` (consonant) cost 2 × 1 = 2. Position 1: `e` and `i` are both vowels, so the cost is 1 + 1 = 2. Both mismatches cost 2, which is a tie, so the earliest position (0) is printed.

Test Cases

Test Case	Input	Output
1	apple apply	1 10 4 10
2	hello jelly	2 13 4 10
3	test test	0 0 -1 0

Test Case	Input	Output
4	aeiou uoiea	4 12 4 5
5	crayon trains	4 39 5 18
6	aemp ximp	2 4 0 2
7	abc xyz	3 17 2 9

Boilerplate Code

Students only need to complete the marked parts.

```
a = input()
b = input()

mismatch_count = 0
total_score = 0

max_position = -1
max_cost = 0

i = 0   # current position

for ch in a:
    # ch is a[i]; compare it with b[i]

    # TODO 1: If ch is different from b[i], this is a mismatch.
    #         Increase mismatch_count.

    # TODO 2: Work out whether ch and b[i] are vowels (True / False).

    # TODO 3: Decide the cost using the rules in the table
    #         (both vowels, both consonants, or one of each).

    # TODO 4: Add the cost to total_score. If the cost is strictly
    #         greater than max_cost, update max_cost and max_position.

    i = i + 1   # move to the next position (already done for you)
```

```
print(mismatch_count)
print(total_score)
print(max_position)
print(max_cost)
```

Complete Solution

```
a = input()
b = input()

mismatch_count = 0
total_score = 0

max_position = -1
max_cost = 0

i = 0

for ch in a:

    bc = b[i]

    if ch != bc:

        mismatch_count = mismatch_count + 1

        a_vowel = ch == "a" or ch == "e" or ch == "i" or ch == "o" or ch == "u"
        b_vowel = bc == "a" or bc == "e" or bc == "i" or bc == "o" or bc == "u"

        if a_vowel and b_vowel:
            cost = i + 1
        elif (not a_vowel) and (not b_vowel):
            cost = 3 * (i + 1)
        else:
            cost = 2 * (i + 1)

        total_score = total_score + cost

        if cost > max_cost:
            max_cost = cost
            max_position = i

    i = i + 1

print(mismatch_count)
print(total_score)
```

```
print(max_position)
print(max_cost)
```

3) Difficult (25 Marks) — Word Statistics

Topics practiced: *input/output, variables, strings (building and indexing), operators and expressions, if/else, for loop, tracking max/min*

Problem Statement

Given a string `s` containing lowercase English letters and spaces, find its words. A **word** is a group of consecutive letters. Words are separated by one or more spaces, and `s` may have extra spaces at the start, at the end, or between words.

Print, each on its own line:

1. The number of words.
2. The longest word (if there is a tie, print the one that appears first).
3. The shortest word (if there is a tie, print the one that appears first).
4. The number of words that start with a vowel (`a`, `e`, `i`, `o`, `u`).

Requirements / Conditions

- `1 <= length of s <= 100`
- `s` contains only lowercase English letters and spaces.
- `s` contains at least one word.
- `.split()` is not allowed.
- The input is read exactly as typed, so spaces at the start and end must **not** be removed.
- Allowed: `input()`, `print()`, variables, `type()`, `id()`, operators and expressions, control structures (`if`/`elif`/`else`, `for`, `while`), strings (indexing and `+`), and lists (`[]` and `.append()`).
- Not allowed: user-defined functions, dictionaries, sets, `len()`, and any string/list method other than `.append()` (for example `.split()`, `.join()`, `.sort()`).
- `print(x, end="")` prints `x` without moving to a new line.

Examples

Example 1

```
Input:
the quick brown fox
Output:
4
quick
the
0
```

Explanation: There are 4 words. `quick` and `brown` both have length 5, so the first one (`quick`) is printed. `the` and `fox` both have length 3, so the first one (`the`) is printed. No word starts with a vowel.

Example 2

```
Input:
  apple   is  an  orange
Output:
4
orange
is
4
```

Explanation: The extra spaces are ignored. The words are `apple`, `is`, `an`, `orange`. `is` and `an` both have length 2, so `is` comes first. All 4 words start with a vowel.

Test Cases

Test Case	Input	Output

1	`the quick brown fox`	`4`
		`quick`
		`the`
		`0`

2	`i love python`	`3`
		`python`
		`i`
		`1`

3	`  apple   is  an  orange`	`4`
		`orange`
		`is`
		`4`

Test Case	Input	Output
4	a bb ccc dd e	5 ccc a 2
5	sun moon star sky	4 moon sun 0
6	one two three four five	5 three one 1
7	z	1 z z 0

Boilerplate Code

Students only need to complete the marked parts.

```
s = input()

word = ""        # the word being built right now
word_len = 0     # its length

count = 0

longest = ""
longest_len = 0

shortest = ""
shortest_len = 101   # bigger than any possible word, so the first word beats it

vowel_start = 0

# The extra space at the end makes sure the last word is also processed.
for ch in s + " ":

    if ch == " ":
        # A space means the current word has ended.
        # Only process it if we actually built a word (skips extra spaces).
        if word_len > 0:

            # TODO 1: Increase the word count.

            # TODO 2: If word_len is greater than longest_len,
```

```
            #         update longest and longest_len.

            # TODO 3: If word_len is less than shortest_len,
            #         update shortest and shortest_len.

            # TODO 4: If the first letter of word (word[0]) is a vowel,
            #         increase vowel_start.

            # Reset for the next word (already done for you)
            word = ""
            word_len = 0

    else:
        # TODO 5: Add ch to the end of word and increase word_len.
        #         (Delete the word "pass" below when you write your code.)
        pass


print(count)
print(longest)
print(shortest)
print(vowel_start)
```

Complete Solution

```
s = input()

word = ""
word_len = 0

count = 0

longest = ""
longest_len = 0

shortest = ""
shortest_len = 101

vowel_start = 0

for ch in s + " ":

    if ch == " ":

        if word_len > 0:

            count = count + 1
```

```
            if word_len > longest_len:
                longest = word
                longest_len = word_len

            if word_len < shortest_len:
                shortest = word
                shortest_len = word_len

            c = word[0]
            if c == "a" or c == "e" or c == "i" or c == "o" or c == "u":
                vowel_start = vowel_start + 1

            word = ""
            word_len = 0

    else:
        word = word + ch
        word_len = word_len + 1

print(count)
print(longest)
print(shortest)
print(vowel_start)
```